

Data Tidying

Getting the data to a workable state

`dplyr` is a grammar
for tidying data.

[Here](#) is a cheatsheet of
commands.

Types of Headaches

Migraine



Hypertension



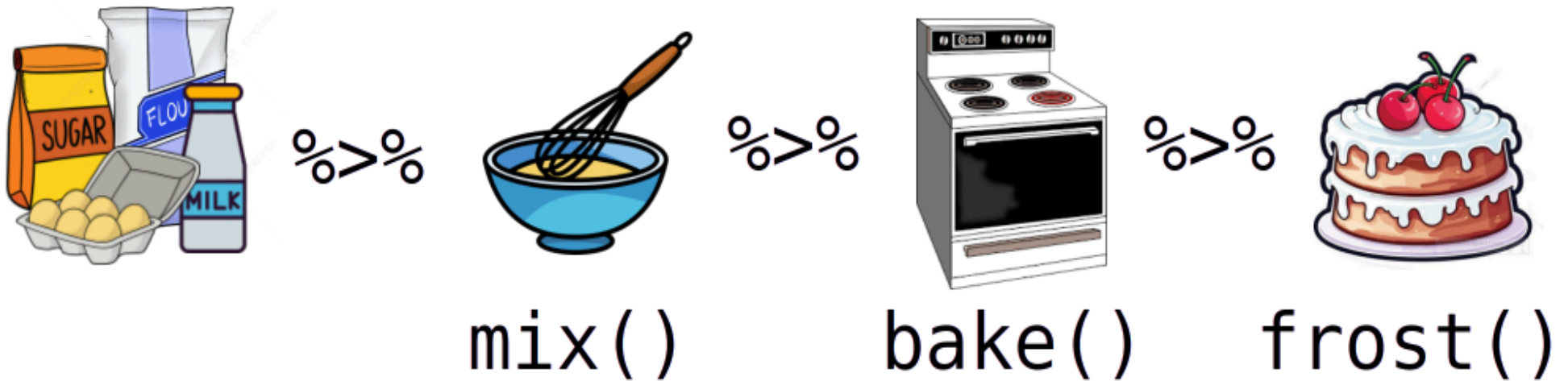
Stress



Handling data issues



The pipe (`%>%`) lets us chain commands together.



Add with **Ctrl/Cmd + Shift + M**

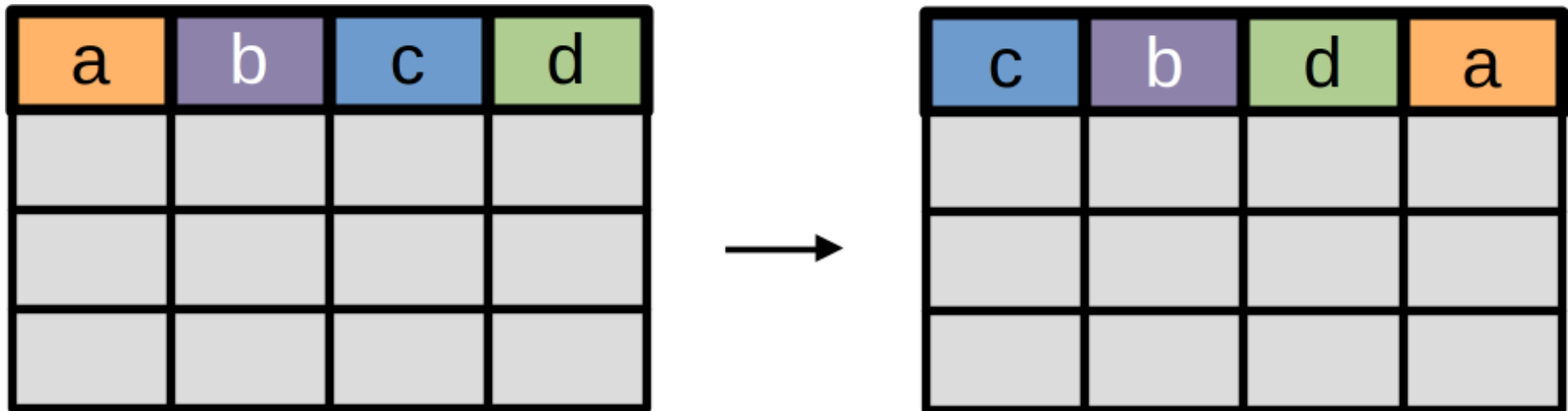
Column commands act on the columns and leave rows unchanged.

- `select()`, `relocate()`, `rename()`, `mutate()`

Row commands act on the rows and leave columns unchanged.

- `arrange()`, `filter()`, `drop_na()`

`relocate()` moves columns.



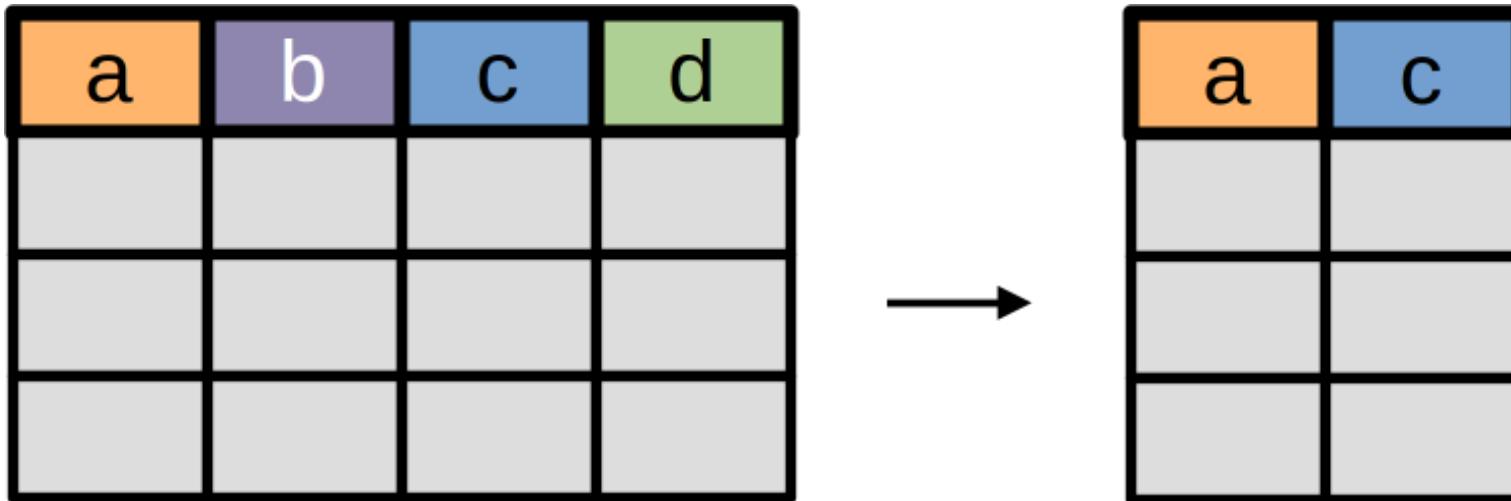
`rename()` renames columns.

a	b	c	d

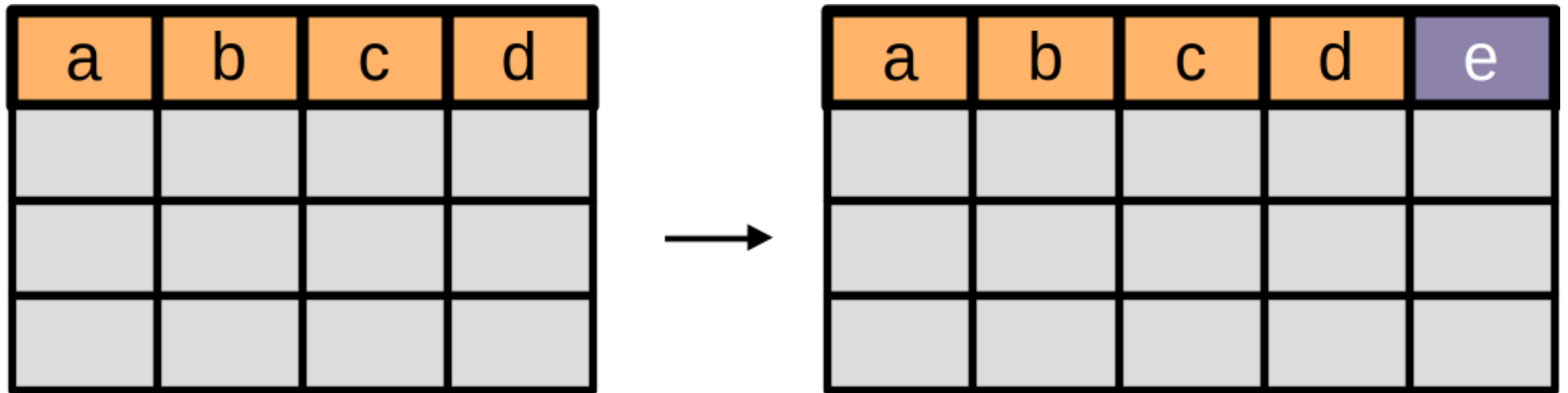


i	ii	iii	iv

`select()` reduces to specific columns.



`mutate()` creates a new column.



Often used with a math equation or with `case_when()`.

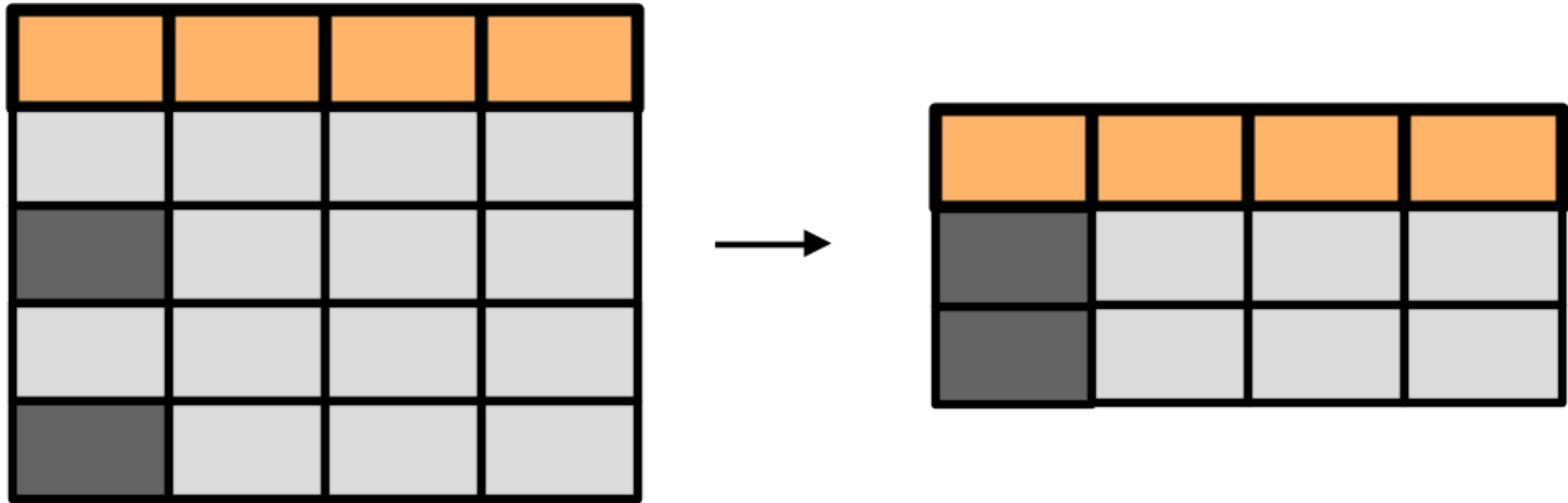
`arrange()` sorts rows.

Orange	Orange	Orange	Orange
Light Gray	Light Gray	Light Gray	Light Gray
Dark Gray	Light Gray	Light Gray	Light Gray
White	Light Gray	Light Gray	Light Gray
Dark Gray	Light Gray	Light Gray	Light Gray



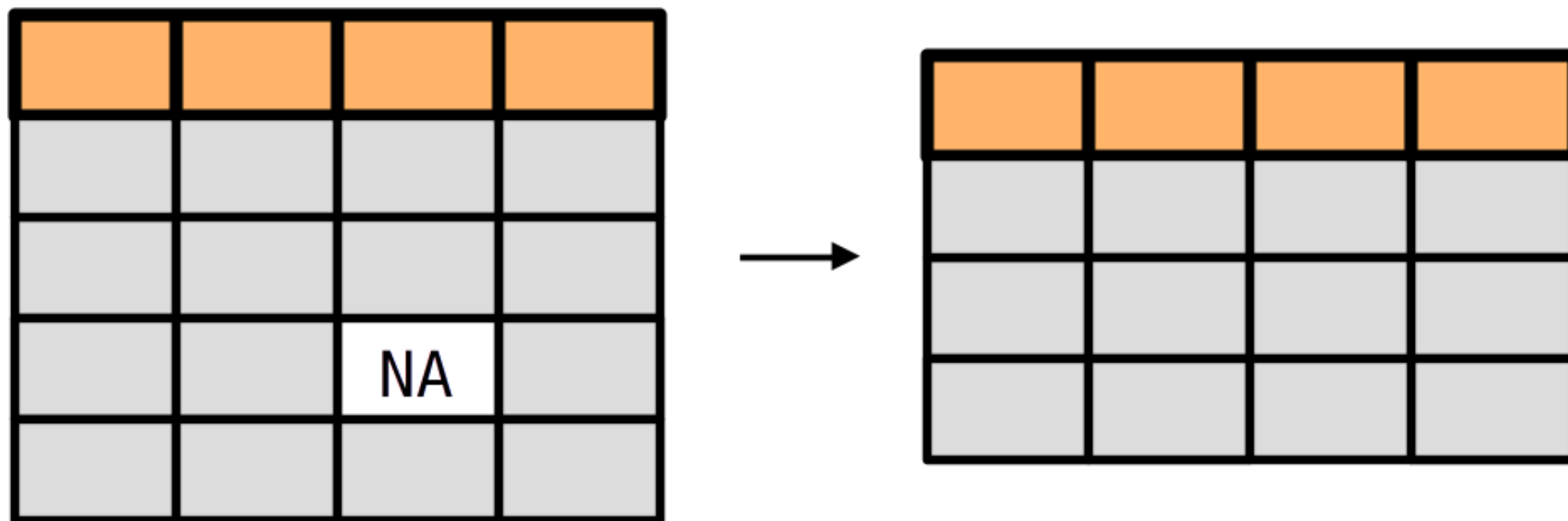
Orange	Orange	Orange	Orange
Dark Gray	Light Gray	Light Gray	Light Gray
Dark Gray	Light Gray	Light Gray	Light Gray
Light Gray	Light Gray	Light Gray	Light Gray
White	Light Gray	Light Gray	Light Gray

`filter()` reduces to specific rows.



Similarly, `slice_min()` and `slice_max()` return rows with the smallest or largest value of a column.

`drop_na()` removes rows with missing values.



Friends A and B are fighting over who is the most die-hard Star Wars fan. Settle the debate with the built-in `starwars` data.

- First, investigate the data. Make sure you're familiar with each column and its type.
- Write R code to resolve each of the arguments.
- Each solution will require a chain of different dplyr commands, connected with a pipe.



`summarize()` reduces a column to a single value.

- e.g. sum, min, mean, max

Combine with `group_by()` for more complex operations.

`summarize():`

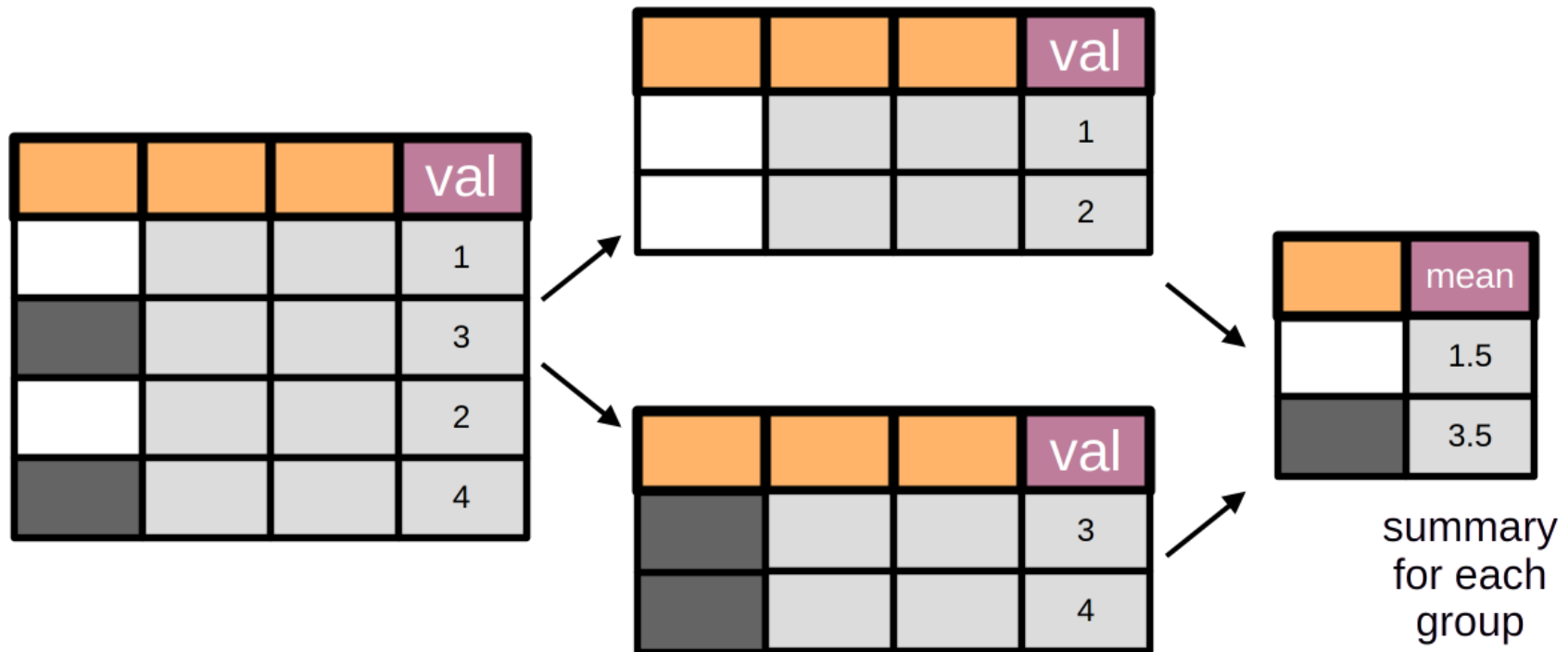
			val
			1
			3
			2
			4



mean
2.5

Single
summary
value

```
summarize() %>% group_by():
```



We can use `mutate()` with `group_by()` for sophisticated column creation.

How does each penguin's weight compare to all the others?

versus

How does each penguin's weight compare to the others of its species?



The volume of a Gentoo or Chinstrap penguin's beak is approximately $(\pi \times r^2 \times \text{length})/2.5$ and the volume of an Adelie penguin's beak is approximately $(\pi \times r^2 \times \text{length})/2$.

- Calculate each penguin's approximate beak volume (pay attention to radius vs depth!)
- Find the penguin with the largest beak volume for each combination of species and island.
- Were most of the winners measured in 2007 or 2009?

`tidyverse` also has commands to join datasets together.



Let's join `grocery_list.csv` and `supermarket.csv`.

Mutating joins append columns together.

- `left, right, full, inner`

Filtering joins keep or remove rows based on the second df.

- `semi, anti`

Both of these lines of code work the same:

```
join(df1, df2)      df1 %>%  
                    join(df2)
```

Also consider the *order* of joining and which df is *prioritized*.

Best practice is to specify the `join_by` argument explicitly.

`join_by` solves issues with:

- Code readability
- Columns with same name but different meanings
- Columns with same meaning but different names

Join the dataframes `drinks` and `coupons` to see which coupons can be used at a user's favorite cafe. Beware of column names!

- Use a mutating join that keeps the user's favorite drinks.
- Be very careful about what columns are being used to join.
- Mutate a new column to calculate each drink's discounted price.



Looking ahead

In [chapter 6](#), we'll learn how to use R for simulation and resampling. If you want a head start, run the following lines of code to investigate the mean flipper length of Adelie penguins:

```
1 adelic <- penguins %>% filter(species == "Adelie")
2 mean(adelic$flipper_length_mm)
3 n <- length(adelic)
4
5 new <- sample_n(adelic, size = n, replace = TRUE)
6 mean(new$flipper_length_mm)
```

What does the object `new` look like? Why does the mean of the “`new`” flipper length change every time you run the code?